Student Record Management System Report

1. Program Design

The Student Record Management System is a menu-driven Python application designed to manage student information. The system stores basic student details such as registration number, name, age, and gender in a CSV file. Additional information including address, contact, and program is stored separately in a JSON file.

The program follows a functional design where each operation is handled by a separate function. The main menu controls user interaction and allows users to select different operations.

2. Key Functions

add_student()

This function collects student information from the user and saves the data into both CSV and JSON files. Input validation is used to ensure that the age is entered correctly.

view_students() ; Reads and displays all student records stored in the CSV file.

search_student() ; Searches for a student using the registration number. If the student does not exist, a custom exception is raised.

update_student() ; Allows users to modify existing student information and saves the updated data back into the CSV file.

delete_student() ; Removes a student record from both CSV and JSON files.

3. Exception Handling Strategy

The system uses Python exception handling to improve reliability and prevent unexpected program crashes.

The program uses:

try block

Contains code that may cause errors, such as:

Reading files, writing files, processing user input and converting data types

except block

Handles different types of errors and displays user-friendly messages such as:

Invalid input missing student records, file reading errors and JSON decoding errors

finally block
This block contains code is always executed regardless of whether an exception occurred or not such as; **Closing files** and l**ogging application completion.**

4. Custom Exceptions:
The program contains multiple custom exceptions:

StudentManagementError: This is the main parent exception for student management errors.

StudentNotFoundError
Used when a user searches for a registration number that does not exist.
Example: Searching for a student with registration number S999 will raise this exception.

DuplicateRegistrationError: Used when attempting to add a student with an already existing registration number.

InvalidStudentDataError: Used when the user enters invalid information such as:

- **Empty fields, incorrect age, invalid email format and invalid contact number.**

These custom exceptions make error handling clearer and easier to maintain.

4. Logging
The logging module is used to record system activities and errors.
The file student_system.log stores: Successful operations, user actions and system errors.
This helps track the behavior of the application.

5. Testing Results
The program was tested using different scenarios:

- Adding a new student: Successful
- Viewing students: Successful
- Searching existing students: Successful
- Searching missing students: Custom exception displayed
- Updating records: Successful
- Deleting records: Successful
- Entering invalid age: Error handled correctly

The system successfully performs all required student management operations.